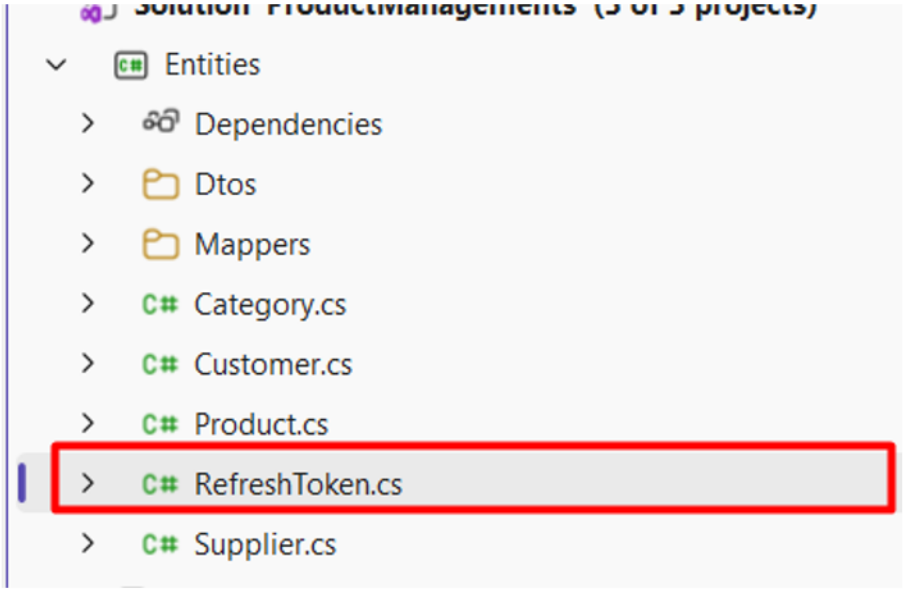
1. បង្កើតmodel ថ្មីដែលមានឈ្មោះថា RefreshToken.cs ដូចរូប




```
public sealed class RefreshToken
{
    [Key]
    public int Id { get; set; }

    [Required]
    public string Token { get; set; } = string.Empty;

    [Required]
    public string UserId { get; set; } = string.Empty;

    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

    public DateTime ExpiresAt { get; set; }

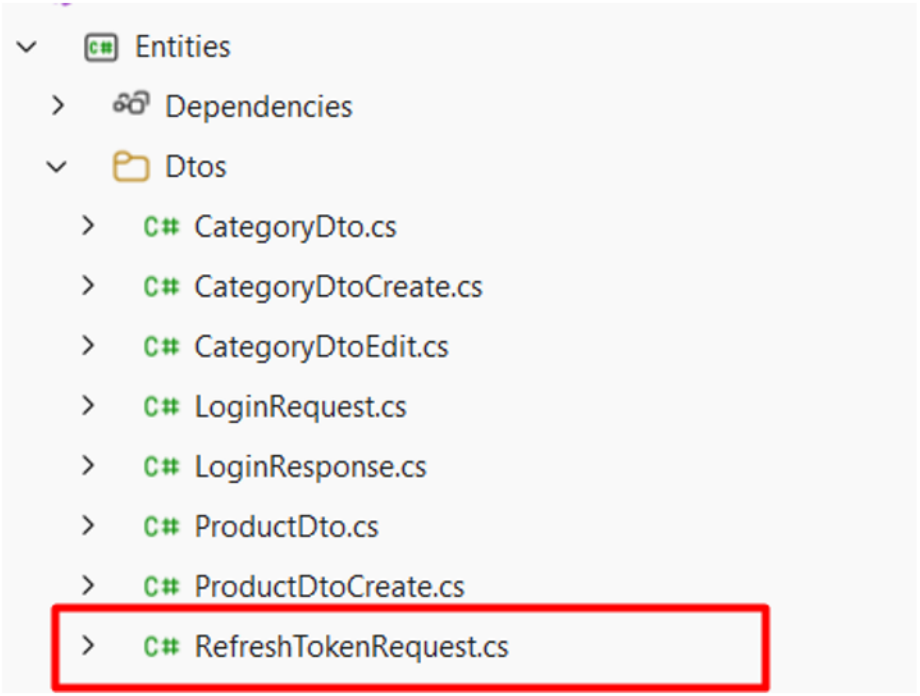
    public DateTime? RevokedAt { get; set; }
```

```
    public bool IsExpired => DateTime.UtcNow >= ExpiresAt;

    public bool IsRevoked => RevokedAt is not null;

    public bool IsActive => !IsExpired && !IsRevoked;
}
```

បង្កើត class RefreshTokenRequest.cs



```
public sealed class RefreshTokenRequest
{
    [Required] public string Token { get; set; } = string.Empty;

    [Required] public string RefreshToken { get; set; } = string.Empty;
}
```

2. Add and Update LogginReponse.cs （Highlight only）

```
public sealed class LoginResponse
{
    public string? UserId { get; set; }
    public string? UserName { get; set; }
    public bool IsSuccess => !string.IsNullOrEmpty(Token);
    public string? Message { get; set; }
    public string? Token { get; set; }

    public string? RefreshToken { get; set; }
    public DateTime? RefreshTokenExpiry { get; set; }

    public static LoginResponse Failure(string message)
    {
        return new LoginResponse
        {
            UserId = null,
            UserName = null,
            Token = null,
            RefreshToken = null,
            Message = message
        };
```

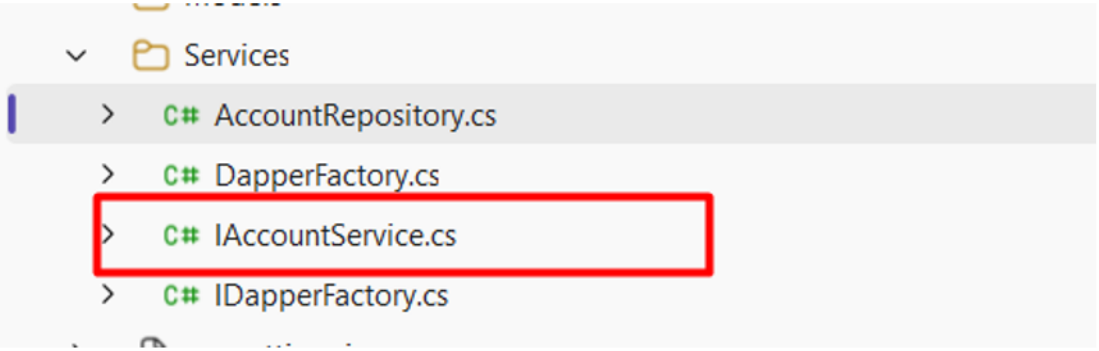
```
    }

    public static LoginResponse UserNotFound()
    {
        return Failure("User not found.");
    }
    public static LoginResponse InvalidCredential()
    {
        return Failure("Invalid Credential.");
    }

    public static LoginResponse InvalidPassword()
    {
        return Failure("Invalid Password.");
    }

    public static LoginResponse Success(string? userId, string? userName, string? token,
string? refreshToken = null, DateTime? refreshTokenExpiry = null, string? message = "Login
successful.")
    {
        return new LoginResponse
        {
            UserId = userId,
            UserName = userName,
            Token = token,
            RefreshToken = refreshToken,
            RefreshTokenExpiry = refreshTokenExpiry,
            Message = message
        };
    }
}
```

3.Add New Method ក្នុង IAccountService.cs ដូចរូប




```
4 references
public interface IAccountService
{
    2 references
    Task<RegisterResponse> CreateUser(RegisterRequest request);
    2 references
    Task<LoginResponse> Login(LoginRequest request);
    2 references
    Task<RoleResponse> CreateRole(string roleName);
    2 references
    Task<RoleDto?> GetRoleById(string id);
    2 references
    Task<LoginResponse> RefreshToken(RefreshTokenRequest request);
    2 references
    Task<bool> RevokeToken(string refreshToken);
}
```

4. Add New Methods ក្នុង AccountRepository.cs

```
    public async Task<LoginResponse> RefreshToken(RefreshTokenRequest request)
    {
        var principal = GetPrincipalFromExpiredToken(request.Token);

        if (principal is null)
            return LoginResponse.Failure("Invalid access token.");

        var userId = principal.FindFirstValue(ClaimTypes.NameIdentifier);
        if (userId is null)
            return LoginResponse.Failure("Invalid access token.");

        var storedToken = await _context.RefreshTokens
            .FirstOrDefaultAsync(rt => rt.Token == request.RefreshToken && rt.UserId ==
userId);

        if (storedToken is null || !storedToken.IsActive)
            return LoginResponse.Failure("Invalid or expired refresh token.");

        // Revoke old refresh token
        storedToken.RevokedAt = DateTime.UtcNow;

        var user = await _userManager.FindByIdAsync(userId);
        if (user is null)
            return LoginResponse.UserNotFound();

        var newAccessToken = await GenerateToken(user);
        var newRefreshToken = await GenerateAndStoreRefreshToken(user.Id);

        await _context.SaveChangesAsync();

        return LoginResponse.Success(user.Id, user.UserName, newAccessToken,
newRefreshToken.Token, newRefreshToken.ExpiresAt);
    }
```

```
public async Task<bool> RevokeToken(string refreshToken)
{
    var storedToken = await _context.RefreshTokens
        .FirstOrDefaultAsync(rt => rt.Token == refreshToken);

    if (storedToken is null || !storedToken.IsActive)
        return false;

    storedToken.RevokedAt = DateTime.UtcNow;
    await _context.SaveChangesAsync();
    return true;
}

private async Task<RefreshToken> GenerateAndStoreRefreshToken(string userId)
{
    var randomBytes = new byte[64];
    using var rng = RandomNumberGenerator.Create();
    rng.GetBytes(randomBytes);

    var refreshToken = new RefreshToken
    {
        Token = Convert.ToBase64String(randomBytes),
        UserId = userId,
        CreatedAt = DateTime.UtcNow,
        ExpiresAt = DateTime.UtcNow.AddDays(7)
    };

    _context.RefreshTokens.Add(refreshToken);

    await _context.SaveChangesAsync();

    return refreshToken;
}

private ClaimsPrincipal? GetPrincipalFromExpiredToken(string token)
```

```
    {
        var options = _configuration.GetSection("Jwt").Get<JwtOption>() ?? new JwtOption();

        var tokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = false,
            ValidateAudience = false,
            ValidateLifetime = false,
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(options.Secret))
        };

        var tokenHandler = new JwtSecurityTokenHandler();
        var principal = tokenHandler.ValidateToken(token, tokenValidationParameters, out var
securityToken);

        if (securityToken is not JwtSecurityToken jwtToken ||
            !jwtToken.Header.Alg.Equals(SecurityAlgorithms.HmacSha256,
StringComparison.InvariantCultureIgnoreCase))
        {
            return null;
        }

        return principal;
    }
```

5.Update Login( ) method នៅក្នុង AccountRepository.cs (Highlight only)

```
    public async Task<LoginResponse> Login(LoginRequest request)
    {
        var user = await _userManager.FindByEmailAsync(request.Email);

        if (user is null) return LoginResponse.UserNotFound();

        if (!await _userManager.CheckPasswordAsync(user, request.Password))
        {
            return LoginResponse.InvalidPassword();
        }

        var token = await GenerateToken(user);

        var refreshToken = await GenerateAndStoreRefreshToken(user.Id);

        return LoginResponse.Success(user.Id, user.UserName, token, refreshToken.Token,
refreshToken.ExpiresAt);
    }
```

6. Add New Methods នៅក្នុង AccountController.cs

```
[HttpPost("refresh-token")]
[ProducesResponseType(typeof(LoginResponse), StatusCodes.Status200OK)]
[ProducesResponseType(typeof(string), StatusCodes.Status400BadRequest)]
public async Task<ActionResult<LoginResponse>> RefreshToken([FromBody] RefreshTokenRequest
request)
{
    var response = await _accountService.RefreshToken(request);

    if (!response.IsSuccess)
```

```
        {
            return BadRequest(response.Message);
        }
        return Ok(response);
    }

    [HttpPost("revoke-token")]
    [ProducesResponseType(StatusCodes.Status200OK)]
    [ProducesResponseType(StatusCodes.Status400BadRequest)]
    public async Task<IActionResult> RevokeToken([FromBody] string refreshToken)
    {
        var result = await _accountService.RevokeToken(refreshToken);

        if (!result)
        {
            return BadRequest("Invalid or already revoked token.");
        }
        return Ok("Token revoked.");
    }
```

7. Run Migration and Update database